

the permissions of a given filename, `sfs_update()` can be used, by passing a NULL pointer for size, zero for `update_ts`, and the pointer to the new permissions as `perms`. Thus, `sfs_chperm()` would be as follows:

```
void sfs_chperm(int sfs_handle, char *fn, int perm) {
    sfs_update(sfs_handle, fn, NULL, 0, &perm);
}
```

Reading a file

Reading a file is basically getting the position of the data blocks from the blocks array of the file's entry, then sequentially reading the contents of the blocks, and displaying that on the `stdout` file descriptor, 1. A couple of things to be taken care of include the following:

- The file is assumed to be without holes, i.e., a block position of 0 in the blocks array indicates no more data blocks for the file.
- Reading should not go beyond the file size. Special care is to be taken while reading the last block with data, as it may be partially valid.

Here's the complete read function, keeping track of valid data using the bytes left to read:

```
uint_t block[SIMULA_FS_BLOCK_SIZE]; // Shared as global with
sfs_write

void sfs_read(int sfs_handle, char *fn) {
    int i, block_i, already_read, rem_to_read, to_read;
    sfs_file_entry_t fe;

    if ((i = sfs_lookup(sfs_handle, fn, &fe)) == -1) {
        printf("File %s doesn't exist\n", fn);
        return;
    }
    already_read = 0;
    rem_to_read = fe.size;
    for (block_i = 0; block_i < SIMULA_FS_DATA_BLOCK_CNT; block_i++) {
        if (!fe.blocks[block_i]) break;
        to_read = (rem_to_read >= sb.block_size) ? sb.block_size
        : rem_to_read;
        lseek(sfs_handle, fe.blocks[block_i] * sb.block_size,
        SEEK_SET);
        read(sfs_handle, block, to_read);
        write(1, block, to_read);
        already_read += to_read;
        rem_to_read -= to_read;
        if (!rem_to_read) break;
    }
}
```

Writing a file

Interestingly, writing is not a trivial function. Getting data

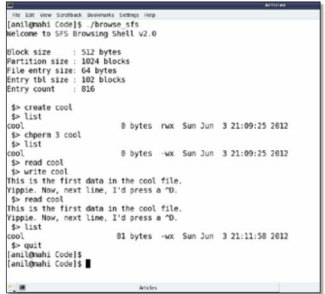


Figure 1: Demo of Simula FS browser's new features

from the user through the browser is fine. But based on that, free available blocks have to be obtained and filled, and then their position noted sequentially in the blocks array of the file's entry. Typically, you do this whenever you have received a block full of data, except the last block. Now, that is tricky—how does one know the last block? Read till the end of the input (marked by Ctrl-D on its own line)—and that is indicated by a return of 0 from read. In that case, check if any non-full block of data is left to be written, and if so, follow the same procedure of obtaining a free available block, filling it up (with the partial data) and updating its position in the blocks array.

After all this, you have finally written the file data, and its data block positions in the blocks array of the file's entry. Now, it's time to update the file's entry with the total size of data written, as well as change the timestamp to the current time. Once done, this entry has to be updated back into the entry table, which is the last step. And in this flow, the following aspects shouldn't be missed out while getting and filling up free blocks:

- Check for no more block positions available in the blocks array of the file's entry.
- Check for no more free blocks available in the FS. In either of these two cases, the intention is to do a graceful stop, with data being written up to the maximum possible, and the rest being discarded. Once again, all of this is put together in the function below:

```
uint_t block[SIMULA_FS_BLOCK_SIZE]; // Shared as global with
sfs_read

void sfs_write(int sfs_handle, char *fn) {
    int i, cur_read_i, to_read, cur_read, total_size, block_i,
    free_i;
```